

BigQuery 팀 접속 안내

팀원에게 그대로 건네는 문서다. 접속 방법과, 틀린 숫자를 뽑지 않기 위해 반드시 지켜야 할 규칙이 들어 있다.
적재하는 쪽 절차는 [ops-bigquery](#) 에 따로 있다.

항목	값
프로젝트	ping-v2-503916 (표시명 ping-v2)
리전	asia-northeast3 (서울)
데이터셋	raw — 40개 표 · 1억 2,373만 행 · 8.74 GiB
권한	팀원 4명 전원 BigQuery 관리자 (2026-08-06 기준)

키 파일은 주고받지 않는다

팀원에게 `credentials.json` 을 전달할 필요가 없다. 콘솔이든 코드든 마찬가지다.

혼동하기 쉬운 지점이라 갈라 둔다.

	무엇	지금 상태
권한	무엇을 할 수 있는가	이미 끝났다. 각자 Gmail 계정에 BigQuery 관리자가 붙어 있다
인증	내가 나임을 어떻게 증명하는가	콘솔은 로그인, 코드는 <code>gcloud</code> 명령 한 번

"코드로 보려면 권한을 더 줘야 한다"는 말은 **틀렸다**. BigQuery 관리자는 이미 이 서비스에서 가장 넓은 역할이고, 코드 접근에 모자란 것은 권한이 아니라 **인증 절차**다. 그 절차가 아래 2번이고, 키 파일이 필요 없다.

`credentials.json` 은 `airflow-loader` 서비스 계정의 비밀키다. 나눠주면:

- 누가 무엇을 했는지 사라진다. 모든 쿼리가 `airflow-loader` 로 기록된다.
- 유출 시 회수 비용이 크다. 키를 무효화하면 **적재 파이프라인이 같이 죽는다**.

- 저장소가 `.gitignore` 로 막아 둔 파일을 손으로 뚫는 셈이 된다.

1. 콘솔에서 SQL — 추가 설정 없음

1. 자기 Gmail 로 <https://console.cloud.google.com/bigquery> 접속
2. 상단 프로젝트 선택기에서 `ping-v2` 선택
3. 왼쪽 탐색기에서 `ping-v2-503916` → `raw` 펼치기 → 표 클릭 → **쿼리**

표 이름을 클릭하면 **스키마·세부정보·미리보기** 탭이 있다.

미리보기는 쿼리 비용이 들지 않는다 — 데이터 생김새만 볼 때는 이쪽을 쓴다.

2. 파이썬 · 주피터에서 — 키 대신 본인 계정 인증

각자 자기 PC 에서 **한 번만** 하면 된다.

① **gcloud CLI 설치** — <https://cloud.google.com/sdk/docs/install>

② **본인 계정으로 인증** (브라우저가 열리면 팀 Gmail 로 로그인)

```
gcloud auth application-default login
```

③ **요금이 청구될 프로젝트 지정**

```
gcloud auth application-default set-quota-project ping-v2-503916
```

④ **파이썬 패키지**

```
pip install google-cloud-bigquery google-cloud-bigquery-storage db-dtypes pandas
```

⑤ **코드에서는 인증 설정이 아예 없다.** ②에서 만든 자격증명을 라이브러리가 알아서 찾는다.

```

from google.cloud import bigquery

bq = bigquery.Client(project="ping-v2-503916", location="asia-northeast3")

df = bq.query("""
    SELECT gender, count(*) AS n
    FROM `ping-v2-503916.raw.app_user`
    WHERE _source = 'local'
    GROUP BY gender
""").to_dataframe()

```

⚠ 이 저장소의 pipeline/ 스크립트는 다르다. 그쪽은 Airflow 가 무인으로 돌아야 해서 서비스 계정 키를 쓴다. 분석에는 그 경로를 쓰지 않는다.

3. Looker Studio · BI 도구

Looker Studio — 새 데이터 소스 → **BigQuery** 커넥터 → 내 프로젝트
 → ping-v2-503916 → raw → 표 선택. 별도 인증 없이 로그인 계정이 쓰인다.

⚠ **BI 도구는 화면을 만질 때마다 다시 스캔한다.** 필터를 한 번 바꿀 때마다 vote_candidate 4 GiB 를 다시 읽으면 무료 한도가 빠르게 준다. 큰 표를 직접 붙이지 말고 집계해 둔 결과를 표로 저장해서 붙인다.

```

CREATE OR REPLACE TABLE `ping-v2-503916.raw.tmp_daily_votes` AS
SELECT date(voted_at, 'Asia/Seoul') d, count(*) n
FROM `ping-v2-503916.raw.vote_item`
WHERE _source = 'local' AND voted_at IS NOT NULL
GROUP BY 1;

```

⚠ 반드시 지킬 쿼리 규칙 넷

전부 오류 없이 틀린 답이 나오는 것들이다. 넷 다 실제로 물렸던 함정이다.

① `_source` 없이 조인하지 않는다

한 표에 **실유저와 합성 데이터가 같이** 들어 있고, 둘 다 id 가 1부터라 **겹친다**.
말이 아니라 실측이다 — 실유저 행은 **사실상 전부** 합성과 id 가 충돌한다.

표	겹치는 id
app_user	23개 (실유저 23명 전원)
vote_item	726개 (전원)
vote_session	78개 (전원)

-- X 실유저와 합성이 조용히 섞인다

```
JOIN raw.app_user u ON v.user_id = u.id
```

-- ✓

```
JOIN raw.app_user u ON v.user_id = u.id AND v._source = u._source
```

실측 — vote_item × app_user 를 위 두 방식으로 세어 봤다.

	행 수
<code>_source</code> 없이	13,459,564
<code>_source</code> 넣고	13,445,278
차이	14,286행이 부풀려진다

오류는 나지 않는다. 그냥 숫자가 커진다.

분석 대상을 먼저 정하고 **모든 표에 같은 `_source` 를 건다**.

합성 데이터 분석이면 `_source = 'local'` , 실유저면 `'supabase'` .

stg 층이 생기면 대리키로 이 실수를 구조적으로 막는다. **아직 없다**.

② `_deleted_at IS NULL` 을 넣는다

raw 는 이력을 지우지 않는다. 원천에서 지워진 행도 **표시만 하고 남는다**.

안 거르면 지운 계정과 그 활동이 오늘 것으로 셈해진다.

③ 시각은 UTC 로 저장돼 있다

KST 로 바꾸지 않으면 시간대 분포가 9시간 밀린다 — 밤 22시 봉우리가 낮 13시로 보인다.

```
extract(hour from voted_at at time zone 'Asia/Seoul')
date(voted_at, 'Asia/Seoul')
```

④ 매출을 셀 때 스텝 결제를 거른다

MVP 의 하트 충전은 결제 없이 들어오는 스텝이다.

```
WHERE store_transaction_id NOT LIKE 'MVP-STUB-%'
```

지금은 스텝 9건이 전부 **실유저**(supabase) 쪽에 있다. 합성 데이터에는 없다.
그래서 **실유저 매출을 볼 때만 문제가 되지만, 실유저 결제가 9건뿐이라**
안 거르면 매출이 통째로 가짜가 된다.

합성 데이터로 할 수 없는 분석이 7가지 있다

질문별 인기·A/B · 후보 슬롯 위치 편향 · 문항 위치별 응답 품질 ·
성별 상호작용 · 게시판 카테고리별 차이 · 앱 버전/플랫폼/기기 ·
광고 네트워크와 시청시간 분포.

설계상 차이가 없게 만들어진 것들이라, 검증하면 반드시 "차이 없음"이 나온다.
차이가 있다고 나오면 그건 발견이 아니라 실수다.

분석 시작 전에 EDA-final-12m-v5.pdf 13장("할 수 있는 것 / 주의 / 없는 것")을
먼저 읽는다. 주의 셋 — 중복 로그 0.4% · 시간대 히트맵은 세션 수로 그리지 말 것 ·
BAN 코호트는 시각으로 자를 것.

비용

저장 10 GiB · 쿼리 1 TiB/월 무료. 결제가 붙어 있어 넘으면 막히지 않고 과금된다.

표	한 번 전체 훑을 때
vote_candidate	4.00 GiB
heart_transaction	1.42 GiB
vote_item	1.28 GiB
vote_received	1.12 GiB
전체 40개 표	8.74 GiB

전체를 훑어도 월 117회분이라 평범하게 쓰면 여유 있다. 다만 습관 두 가지만:

- **SELECT * 를 쓰지 않는다.** BigQuery 는 열 단위로 읽어서, 필요한 열만 적으면 스캔량이 그만큼 줄어든다. **WHERE** 로는 안 줄어든다.
- 콘솔은 쿼리를 치면 실행 전에 **"이 쿼리는 N GB 를 처리합니다"** 를 오른쪽 위에 보여준다. 큰 표를 만질 때 그 숫자를 보고 실행한다.
- 탐색 중에는 **LIMIT** 대신 **TABLESAMPLE SYSTEM (1 PERCENT)** 를 쓴다.
LIMIT 은 스캔량을 줄이지 못한다.

사소하지만 자주 걸리는 것 — **컬럼 별칭에 한글을 쓰려면 백틱으로 감싼다.**

AS 건수 는 문법 오류고 **AS `건수`** 는 된다.

팀원에게 주지 않는 것

- **data/personas.json** — 합성 데이터의 **정답지**(페르소나 라벨)다.
분석자가 보면 분석 연습이 성립하지 않는다.
- **credentials.json** — 위에 적은 대로.

지금 팀원은 표를 지울 수 있다

전원이 **BigQuery 관리자**라 데이터셋·표 삭제와 적재 설정 변경이 가능하다.

의도한 선택이지만, **raw** 를 날리면 **1억 2,369만 행을 다시 올려야 하고 2시간이 든다** (로컬 **pgtest** 에 원본이 남아 있어 복구 자체는 가능하다).

작업용 표는 tmp_ 접두어로 만들고, 접두어 없는 표는 지우지 않는다.

읽기만 필요한 사람은 BigQuery 데이터 뷰어 + BigQuery 작업 사용자 조합으로
낮출 수 있다 — 분석·쿼리는 그대로 다 된다.

[적재 쪽 절차](#) · [CLAUDE.md](#) 로 돌아가기